JavaScript Data Flow Guide For This Journal App

Project references
- templates/page.html lines 103, 129, 144, 157, 165, 205, 226, 254, 295, 299
- journals/serializers.py
- journals/models.py

1. Big picture

The JavaScript in templates/page.html exists to connect three layers:
1) the browser UI,
2) the HTTP API,
3) Django database saving.

The browser does not save journals to the database by itself. It only collects values from the page and sends them to Django. Django and Django REST Framework validate the incoming data and save it through the serializer and model.

The flow for saving is:
- The user types a title, picks a date, and writes content in Quill.
- saveData() reads those values from the DOM.
- toIsoStringFromLocalInput() converts the date input to a standard ISO string.
- JSON.stringify() turns the JavaScript object into JSON text.
- fetch('/api/journals/', { method: 'POST', ... }) sends the request to Django.
- The serializer parses journal_date as a DateTimeField.
- The model stores the row in the database.

The flow for reading is:
- loadJournals() sends a GET request to /api/journals/.
- Django returns only journals for the logged-in user.
- renderJournalList() turns that response into clickable journal items.
- Clicking one item loads its values back into the editor.

2. Why JSON is used here

JSON is not the database. JSON is just the transport format used between JavaScript and Django over HTTP.

Without JSON, the frontend would still need some format to send structured data like:
- journal_title
- journal_description
- journal_date

JSON is a good fit because:
- browsers work with JavaScript objects naturally,
- Django REST Framework parses JSON well,
- nested and structured data stays readable,
- it is the normal format for APIs.

So the rule is:

- JavaScript creates an object,
- JSON carries that object over the network,
- Django saves it into models.

3. Function by function explanation

getCookie(name) in page.html line 103
Why it exists:
- Django protects POST and DELETE requests with CSRF.
- If the frontend sends data without the CSRF token, Django may reject the request.

How it works:
- It reads document.cookie.
- It finds the cookie named csrftoken.
- It returns the token so fetch() can send it in the headers.

Why this matters for data:
- It is part of secure writing, not part of journal content itself.
- The save and delete requests depend on it.

toDateTimeLocalValue(dateValue) in page.html line 129
Why it exists:
- A datetime-local input expects a local browser-style value like YYYY-MM-DDTHH:mm.
- The API usually returns an ISO datetime.
- Those two formats are related but not identical in usage.

How it works:
- It builds a Date from the API value.
- It shifts by the browser timezone offset.
- It returns a string trimmed to minutes so the datetime-local input can display it.

Why this matters for data:
- Without conversion, the saved timestamp may display incorrectly when a journal is reopened.
- This function is about display formatting, not database persistence.

toIsoStringFromLocalInput(dateValue) in page.html line 144
Why it exists:
- HTML datetime-local gives a local date and time with no timezone marker.
- APIs and backends are usually happier with a standard ISO 8601 timestamp.
- If the user leaves the field blank, you wanted the current date and time to be sent.

How it works:
- If the field is empty, it returns new Date().toISOString().
- If the field has a value, it creates a Date object from that local value.
- It then returns date.toISOString(), which produces UTC-based ISO text.

Why this matters for data:
- It standardizes the payload before it leaves the browser.
- Django REST Framework can parse ISO datetimes cleanly.

- This reduces ambiguity between browser-local time and server time.

Important note:
- toISOString() always produces UTC with a trailing Z.
- That means the stored value may look different from the local clock string, but it still represents the same moment in time.

resetEditor() in page.html line 157
Why it exists:
- The app needs a clean state when the user starts a new journal.

How it works:
- selectedJournalId becomes null.
- The title and date inputs are cleared.
- Quill content is reset.
- Focus moves back into the editor.

Why this matters for data:
- It prevents accidental reuse of the previous journal values.
- It makes the UI state match the user’s intention to create a fresh entry.

renderJournalList(journals) in page.html line 165
Why it exists:
- The API returns data objects, but users need clickable UI.

How it works:
- It maps each journal object into sidebar HTML.
- It attaches click listeners to open buttons.
- It attaches click listeners to delete buttons.
- On open, it copies the selected journal data into the title, date, and editor fields.

Why this matters for data:
- It is the bridge from JSON response data back into visible UI state.
- It also keeps selectedJournalId so the page knows which journal is being viewed.

loadJournals(searchValue = ’’) in page.html line 205
Why it exists:
- The page should load journals when it opens and refresh the list after saves or deletes.

How it works:
- It builds the URL.
- If there is search text, it adds ?search=...
- It uses fetch() to request the journals list.
- It converts the response with response.json().
- It hands the parsed array to renderJournalList().

Why this matters for data:
- It is the read path from Django to the browser.
- This is where the app asks the server for the current user’s data.

deleteJournal(journalId) in page.html line 226
Why it exists:
- Users need a way to remove saved journals.

How it works:
- It asks for confirmation.
- It sends a DELETE request to /api/journals/<id>/.
- If the deleted item is currently open, it clears the editor.
- It reloads the list.

Why this matters for data:
- It is a write operation against the backend.
- The frontend asks; Django actually performs the deletion.

saveData() in page.html line 254
Why it exists:
- This is the main save pipeline.

How it works:
- Reads title from #journal_title.
- Reads date from #journal_date.
- Reads HTML content from quill.root.innerHTML.
- Validates that title is not empty.
- Builds a plain JavaScript object.
- Converts it into JSON with JSON.stringify().
- Sends it to /api/journals/ using fetch().
- Parses the response JSON.
- Reloads the sidebar list after success.

Why this matters for data:
- It is the central handoff point from browser state to server persistence.
- It decides what exact payload is sent.

The payload shape is essentially:
```
{
  "journal_title": "...",
  "journal_description": "<p>...</p>",
  "journal_date": "2026-04-08T09:30:00.000Z"
}
```

searchInput.addEventListener('input', ...) in page.html line 295
Why it exists:
- Search should feel live instead of requiring a submit button.

How it works:
- Every time the input changes, loadJournals() is called with the current text.

Why this matters for data:
- It changes which subset of server data is requested and displayed.

createEntryButton.addEventListener('click', resetEditor) in page.html line 299
Why it exists:
- Clicking the plus button should start a new entry immediately.

How it works:
- It attaches the click event directly to the button.
- When clicked, resetEditor() runs.

Why this matters for data:
- It changes frontend state only.
- It does not save until saveData() sends a request.

4. How the data should ideally be handled

Good separation looks like this:
- Frontend responsibility: collect input, present UI, send requests, render responses.
- Serializer responsibility: validate and normalize incoming data.
- Model responsibility: define the database shape and defaults.
- View responsibility: connect request, serializer, permissions, and queryset filtering.

That is the reason your current stack is reasonable:
- JavaScript should not write to SQLite or PostgreSQL directly.
- Django should stay responsible for database writes.

5. Why I changed the date handling the way I did

Earlier, the model used auto_now_add or a server-side always-now behavior. That caused user-selected dates to be ignored.

The improved idea is:
- If the frontend sends a date, keep it.
- If the frontend sends nothing, fall back to now.

This is cleaner because:
- the user can intentionally choose a journal date,
- empty input still works,
- the serializer can parse ISO 8601 cleanly,
- the model keeps a safe default.

This is also close to real-world practice:
- clients send structured values,
- servers validate them,
- servers apply defaults if fields are missing.

6. Design notes and tradeoffs

Why use fetch()?
- It is the built-in browser API for HTTP requests.
- It keeps the frontend lightweight.

Why use response.json()?
- The API returns JSON.
- The browser must parse that response text into JavaScript values before the page can use it.

Why use addEventListener()?
- It keeps behavior attached to events clearly.
- It is more flexible than old inline event patterns.

Why use helper functions instead of doing everything inside saveData()?
- Smaller functions are easier to debug.
- Date formatting logic can be reused.
- The code becomes easier to reason about.

7. Things to improve next in this app

Good next improvements:
- Use PUT or PATCH when editing an existing journal instead of always POSTing a new one.
- Store Quill Delta as well if you need richer editor history.
- Show created_at and updated_at separately from the user-chosen journal date.
- Add better error messages under form fields instead of only alert().
- Convert the inline script into a separate static JavaScript file for maintainability.
- Add debouncing to search so it does not fetch on every single keystroke immediately.

8. Learning roadmap

Learn in this order:
1) JavaScript basics: objects, functions, arrays, events.
2) The DOM: query selectors, input values, innerHTML, event listeners.
3) HTTP basics: GET, POST, DELETE, headers, status codes.
4) JSON: stringify and parse.
5) Dates and timezones: local time, UTC, ISO 8601.
6) Django REST Framework serializers and viewsets.
7) Authentication, sessions, and CSRF.

9. Small projects to build

Project 1: Notes app
- Title and body fields.
- Save notes with fetch().
- List notes from the backend.
- Delete notes.

Project 2: Task manager
- Task text, due date, completed flag.
- Filter by completed or pending.
- Practice JSON payloads and form handling.

Project 3: Expense tracker
- Amount, category, note, datetime.

- Learn date parsing and totals.

Project 4: Movie watchlist
- Title, status, rating, watched_at.
- Practice update versus create flows.

Project 5: Journal with version history
- Save a journal.
- Edit it with PATCH.
- Add a JournalVersion model to store old copies.

10. Official resources

MDN Fetch API:
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

MDN JSON.stringify():
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

MDN Date:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

MDN Date.prototype.toISOString():
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toISOString

MDN input datetime-local:
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/datetime-local

MDN addEventListener():
https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

Django REST Framework serializer fields:
https://www.django-rest-framework.org/api-guide/fields/

Django REST Framework serializers:
https://www.django-rest-framework.org/api-guide/serializers/

Django model field reference:
https://docs.djangoproject.com/en/5.2/ref/models/fields/

Django CSRF protection:
https://docs.djangoproject.com/en/5.2/ref/csrf/

11. Final takeaway

The JavaScript in this page is not replacing Django. It is preparing data, sending it safely, and rendering server responses. The backend is still the source of truth for validation, ownership, permissions, and database saving.